

Physics of a Planetary Renderer

Models, approximations, and alternative approaches in PlanetSimulation

Implementation study · version 0.0.1

Abstract. PlanetSimulation couples terrain, prescribed orbital motion, reflected illumination, and an optically active atmosphere in an interactive renderer. This paper explains the physical ideas behind those models and the approximations that make them practical. Particular attention goes to atmospheric density, molecular and aerosol extinction, single scattering, twilight visibility, and refractive ray curvature. For each major subsystem, we compare the implemented method with a plausible alternative and state what physical behavior is retained or lost. Twelve generated vector diagrams explain the algorithms; versioned captures show actual rendering. The reported GPU benchmark comes from the project README and is qualified by its original workload and hardware. The model is intentionally limited: it does not solve climate, fluid flow, multiple atmospheric scattering, or mutual N-body gravity.

Keywords: real-time graphics · procedural terrain · Keplerian dynamics · atmospheric optics · adaptive rendering · reproducible engineering

1. Scope and physical questions

The paper describes the current source and its reproducible images, not a claim of physical fidelity for every planet. The useful questions are causal: what determines a surface point, what gravitational model moves the bodies, what radiance reaches the eye, and which terms can be approximated without destroying the intended visual behavior? Equations are simplified descriptions of the implementation; where a coefficient is tuned for the miniature scene, that is stated explicitly. Images come from `docs/screenshots/`, with capture inputs and hashes in `docs/captures/generation.json`. Diagrams in `figures/` are explanatory schematics generated by `make_figures.py`.

The atmosphere deserves special care. A convincing sunset depends on the length and density of the optical path, not merely on a sky color. The renderer therefore keeps absorption, out-scattering, in-scattering, and geometric visibility as separate quantities. Its reference pressure and temperature are configured inputs; no weather or climate equation evolves them. Tests and controlled captures check specific invariants and regressions, but an attractive image alone cannot establish that the numerical model is physically accurate.

2. Representing a planet in a camera

2.1. From configuration to a projected sphere

A position in a planet's local mesh is transformed by a model matrix, a camera view matrix, and a projection matrix. In the project's column-vector convention,

$$x_{\text{clip}} = PVMx_{\text{local}}, \quad x_{\text{ndc}} = \frac{x_{\text{clip}}}{w_{\text{clip}}}.$$

This mapping establishes which physical coordinates each calculation uses. Heights belong to the rotating planet frame; orbital positions and light directions belong to the world frame; depth and pixel reconstruction belong to the camera frame. Mixing them can make a mountain move under a stationary observer or make a shadow follow the camera. GLM supplies the matrix operations, while behavior tests check the resulting projections. Figure 1 records the data path.

From a configured body to a framebuffer pixel



Figure 1: The foundational transform and shading path. $P \cdot V \cdot M$ follows the current shader convention; the final display transform happens after linear-light composition. Generated schematic.

A hidden-window `--render-test` and PNG smoke test make camera framing, framebuffer readback, and image output observable in headless environments. Figure 2 is an actual capture of a compact regression scene. It is evidence of the current renderer at a fixed input state.

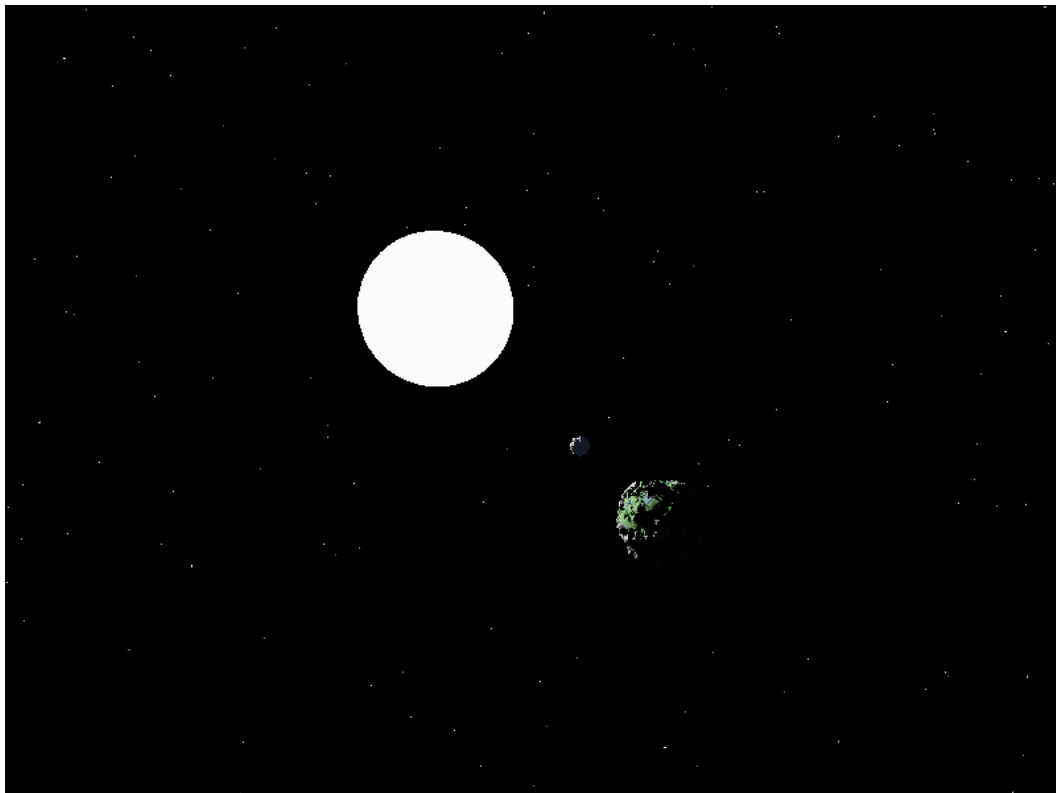


Figure 2: Headless rendering of the compact scenario. It checks the projected positions of the bodies at a reproducible input state.
Source: `docs/screenshots/solar-view.png`; provenance: `docs/captures/generation.json`.

2.2. A reproducible scene as an experimental unit

The JSON parser performs type and range checks before GPU resources are replaced. Scene replay stores a resolved scenario, camera, time, and quality choices next to a PNG. The image is an observation; its sidcar specifies a repeatable input state. On live reload, a new CPU scene is staged before the current one is exchanged, so an invalid or half-written file retains the last valid scene. This matters for physical comparison: changing the camera or exposure at the same time as atmospheric density would confound the result.

3. Terrain as a sampled physical surface

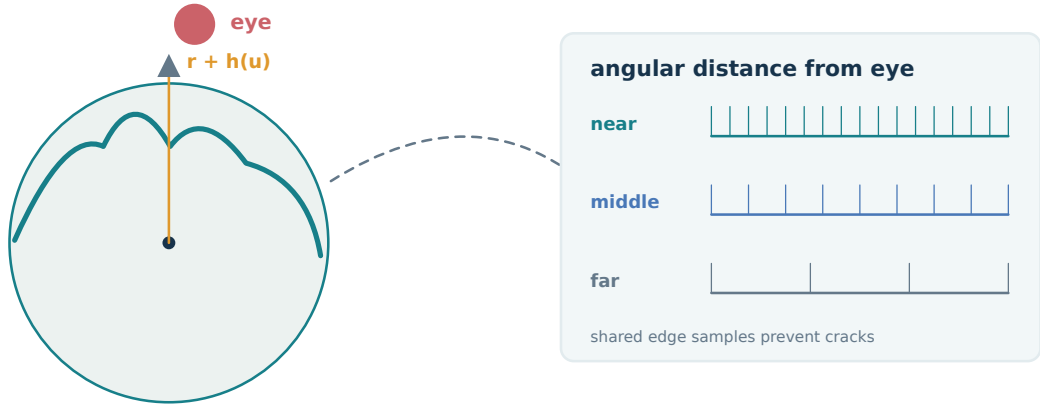
3.1. Procedural surface as a camera-independent function

The surface height is evaluated in a normalized planet-local radial direction u . A simplified form is

$$h(u) = h_{\text{landscape}(u)} + \sum_i A_i f_i(u; \text{seed}_i, \text{frequency}_i, \text{octaves}_i).$$

The implemented functions include value-noise fBm and a ridged variant. Broad continent and cliff fields determine large-scale shape; finer octaves add local detail. This is synthetic topography: it has spatial correlation and multiple length scales but no tectonics, erosion, or material transport. The seed and local coordinate frame make repeated samples of the same point deterministic. Since terrain geometry is rebuilt as the camera moves, a view-dependent height function would make the ground itself rise and fall. The current code samples `heightAt(radial)` during every LOD rebuild. Figure 3 distinguishes the fixed field from its variable triangulation.

Stable height field and camera-dependent tessellation



(u) is fixed in planet-local coordinates; only mesh density follows the camera.

Figure 3: The height field is fixed in planet coordinates; near, middle, and far zones change only sampling density. Bars are illustrative edge densities, not an image of the actual mesh. Generated schematic.

The sphere begins with 20 icosahedral faces, subdivided twice into 320 stable base faces. For nearby views, each base face is assigned one of three edge-segment zones using its angular distance to the eye. A previous-zone hysteresis margin prevents repeated switching at a threshold. Steep faces can receive extra segments if the triangle budget permits. Edges shared by unequal zones are sampled once at the finer requirement, then reused on both sides; concentric interior rings fill the faces without T-junctions. The budget loop first removes optional steep refinement and then lowers less-important zones. This preserves one continuous sampled surface under a per-planet triangle ceiling.

An alternative is a cube-sphere quadtree with screen-space error bounds. It can allocate triangles more directly to projected error and support very large planets, but needs explicit seam and level-transition rules. A fixed high-resolution sphere avoids LOD transitions at the cost of drawing distant detail that cannot affect the image. Here the icosahedral base and fixed height query suit a small interactive scene where local walking must remain stable.

The sampled height feeds normals and terrain color factors as well as eye clearance. Water is a separate spherical shell, an approximation that omits tides and waves in the body's shape. The local camera follows the planet's translation and rotation through a planet-fixed frame, so the same latitude/longitude refers to the same terrain location after orbital motion or spin. Walking near a pole uses a great-circle step and parallel transport of the view basis; longitude itself becomes singular there. Figure 4 illustrates why transporting the basis in 3-D is better behaved than rebuilding it from longitude after every step.

Great-circle camera transport across a pole

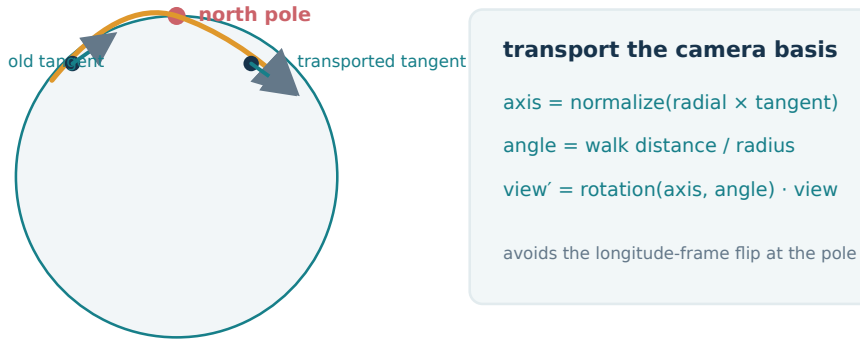


Figure 4: Great-circle position update and basis transport across the north pole. A longitude-derived frame can flip at the pole; a transported 3-D basis remains continuous. Generated schematic.

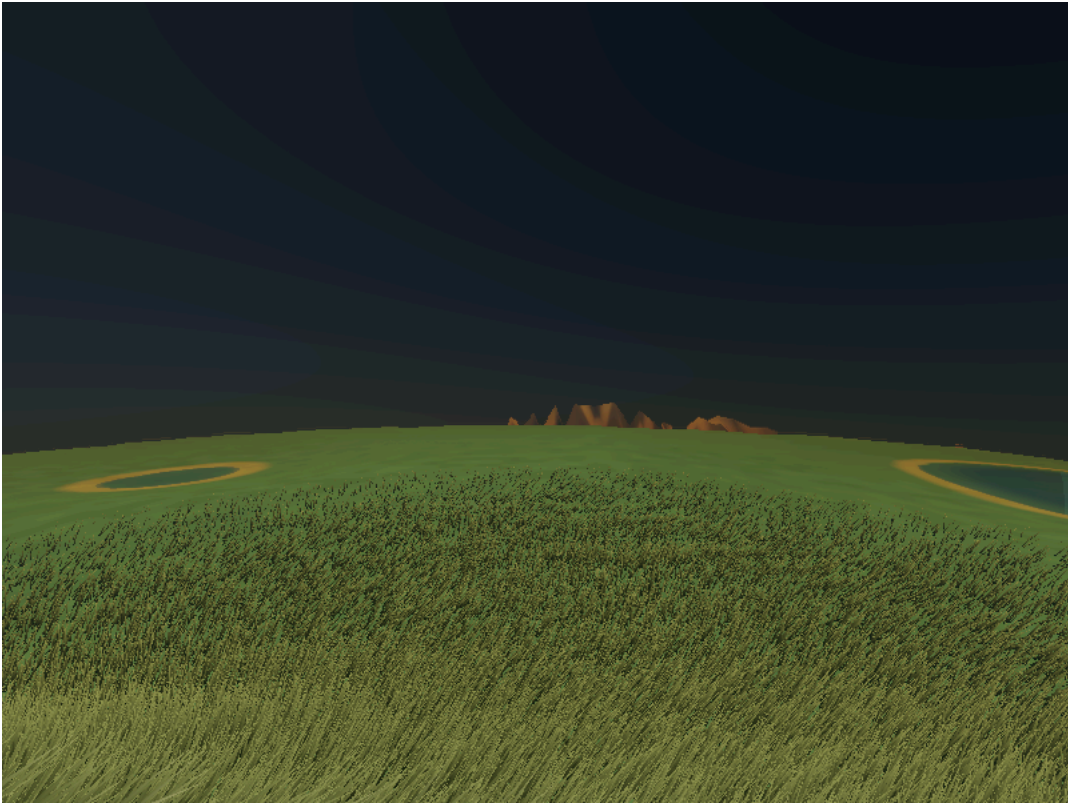


Figure 5: Surface-camera capture of procedurally generated terrain. Geometry derives from the stable height function while tessellation is selected for this camera. Source: current gallery.

4. Orbital motion and light transport

4.1. Prescribed Kepler ellipses with collective mass

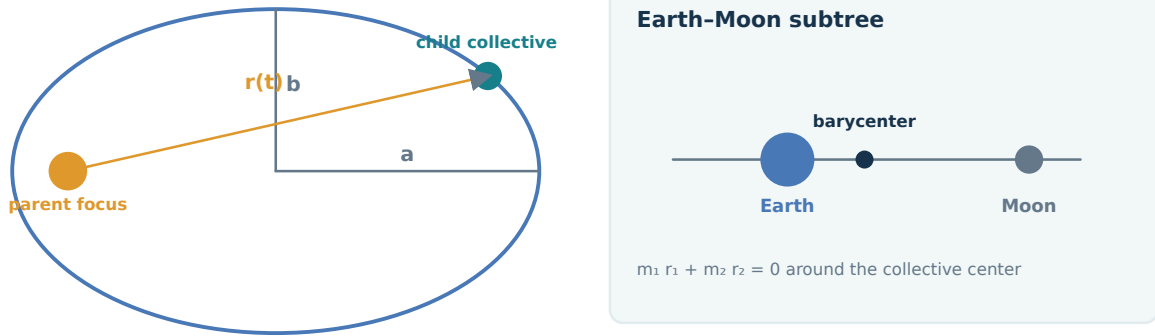
A body names its parent and supplies semi-major and semi-minor axes a, b , a mean anomaly, inclination, ascending node, and periapsis orientation. The two-body approximation gives eccentricity and period as

$$e = \sqrt{1 - \frac{b^2}{a^2}}, \quad \mu = G(m_{\text{parent}} + m_{\text{collective}}), \quad T = 2\pi\sqrt{\frac{a^3}{\mu}}.$$

Here a and b are converted from the scenario's distance unit to metres before applying the SI gravitational constant; the code performs the equivalent calculation in world units. At time t , mean anomaly advances linearly, representing equal areas swept in equal times. A bounded bracketing solve obtains eccentric anomaly E from $M = E - e \sin E$, and the orbital-plane position is $(a(\cos E - e), b \sin E)$. Three rotations orient that ellipse in the world. The implementation uses a rationalized form near extreme periapsis to avoid cancellation.

Evaluating at absolute time avoids frame-rate-dependent drift and makes pausing, replay, and speed changes deterministic.

Kepler ellipse and hierarchical barycentric recoil



Solve $E - e \sin E = M(t)$; then rotate the orbital-plane ellipse into world space.

Figure 6: An ellipse is specified relative to a focus; a child subtree and its parent recoil around a mass-weighted center. The right panel abstracts the Earth–Moon relation; sizes and offsets are not to scale. Generated schematic.

Each subtree acts as a collective point mass in its parent’s prescribed ellipse; children induce mass-weighted recoil of the parent. Thus an Earth–Moon pair has an internal barycenter, and the Sun moves in response to its descendants while its configured position remains the time-zero anchor. This retains the main geometric and mass-weighted effects while omitting inter-branch perturbations and tidal evolution. A symplectic N-body integrator would capture mutual perturbations and long-term exchange of energy and angular momentum, but would need initial velocities, a suitable time step, and careful treatment of close encounters. Prescribed ellipses are a good fit for deterministic scene replay, not for predicting orbital evolution.

For the 0 overlay, the application samples the complete hierarchy at 321 times for each body: ten revolutions at 32 segments per revolution. A moon’s inertial trail must inherit its parent’s translation rather than drawing one stationary local ellipse. Figure 7 shows that distinction.

Ten future moon revolutions inherit parent translation

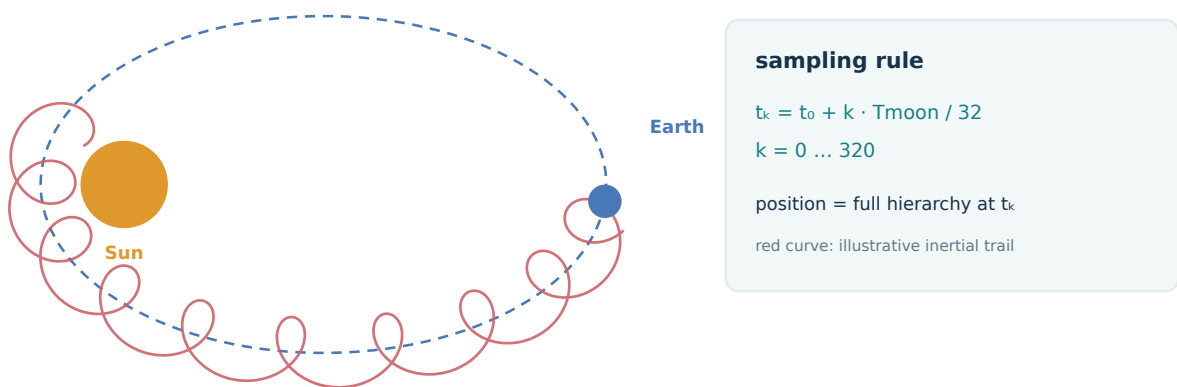


Figure 7: Illustrative moon trail in the inertial world frame. The repeated local cycles travel with the parent; the curve is explanatory and does not plot a measured scenario trajectory. Generated schematic.

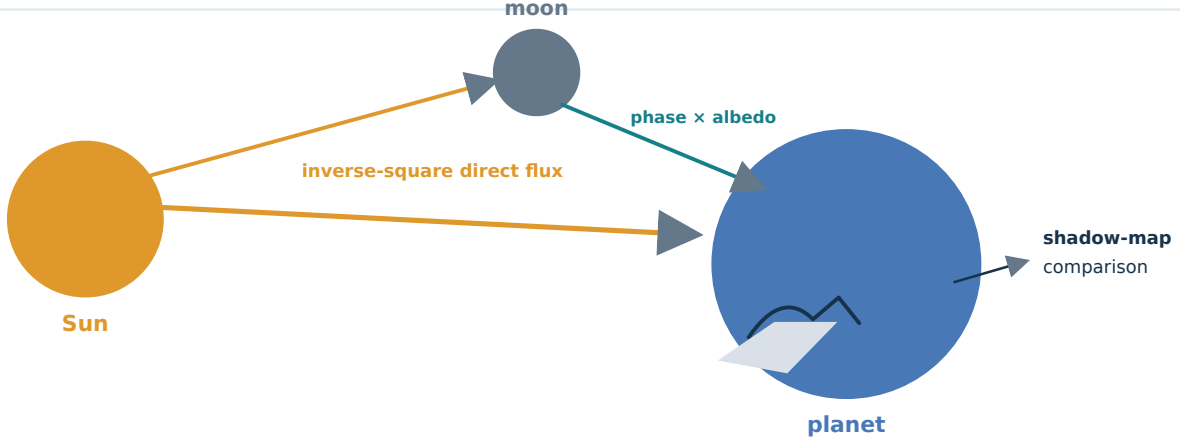
4.2. Light transport before atmospheric composition

Per-frame CPU lighting computes direct flux with inverse-square distance and reflected flux with geometric albedo, a diffuse-sphere phase function, and inverse-square reflector-to-receiver falloff. Sun luminosity is derived from absolute bolometric magnitude using the IAU zero point. The phase law matters: a full moon presents more of its illuminated hemisphere to the receiver than a crescent. Shaders sum direct, reflected, and indirect

contributions in linear RGB before an exponential display shoulder and sRGB encoding. Summing encoded display values would break the proportional meaning of flux and exposure.

A directional orthographic depth map is made per planet from the local Sun direction. The same map constrains opaque terrain, water, and direct atmospheric scattering. Its depth test approximates geometric occlusion by terrain; it does not solve light diffraction or soft penumbrae from the full solar disk. The map is cached until local Sun direction, terrain revision, extent, or resolution changes enough to alter its result. Ray tracing each point toward the Sun would give a more direct visibility test and potentially better soft shadows, but its cost is high for every surface fragment and atmospheric sample. Figure 8 separates the direct and reflected paths.

Direct illumination, reflected moonlight and terrain visibility



The same terrain depth map gates opaque shading, water reflection and direct atmospheric scattering.

Figure 8: The Sun contributes direct flux to each body, the Moon reflects a phase-dependent portion toward the planet, and terrain visibility controls direct light. The diagram is schematic; arrow widths do not encode flux.

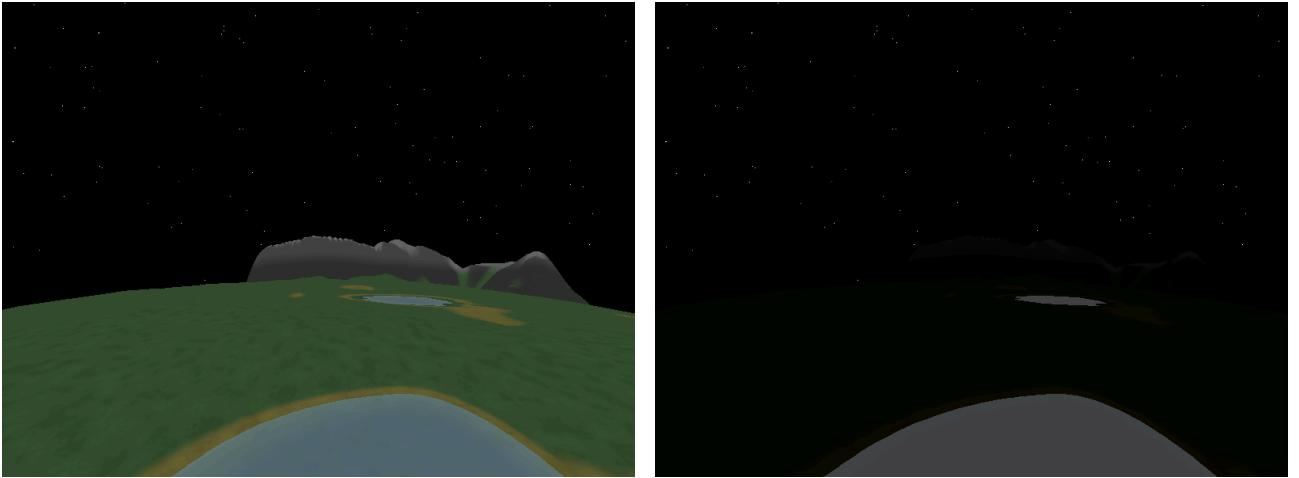


Figure 9: A controlled night-scene comparison from the current gallery: moonlit (left) and moonless (right). The saved replays preserve camera and lighting inputs, allowing the reflected-light term to be isolated.

5. Atmospheric optics: from air state to pixel radiance

5.1. Why density is modeled as a vertical profile

In an approximately isothermal atmosphere under roughly constant gravity, hydrostatic balance and the ideal gas law produce an exponential molecular density with scale height $H \approx k_B \frac{T}{mg}$ (R6). PlanetSimulation uses the same shape for molecules and a separate, usually shorter shape for aerosols:

$$\rho_{m(h)} = \exp\left(-\frac{h}{H_m}\right), \quad \rho_{a(h)} = \exp\left(-\frac{h}{H_a}\right).$$

This is a physically motivated shape, but the actual scale heights are set to fractions of the configured shell thickness, not calculated from local gravity and temperature. That choice keeps the visible atmosphere proportionate to a small rendered planet. Both densities taper smoothly to zero over the outer 15% of the shell

so the finite numerical boundary does not appear as a hard optical edge. A hydrostatic solver with altitude-dependent temperature would produce more defensible densities, especially for a very deep atmosphere; it would also need gravity, thermal state, and boundary conditions that the current scene format does not supply.

At the reference surface, pressure and temperature scale molecular density approximately as $\frac{P}{T}$. Gas mixture changes the reference refractivity and Rayleigh strength. The code samples Rayleigh coefficients at three representative visible wavelengths and rescales their reference column from an 8 km Earth-like value into units of planet radii. This preserves recognizable color separation, including stronger short-wave scattering, but is not a spectral atmosphere prediction for an arbitrary small world. A spectral solver would resolve wavelength-dependent gas and aerosol properties more faithfully, then integrate to display RGB; it would require more storage and sampling per pixel.

5.2. Extinction, scattering, and the color of the sky

Molecular scattering, aerosol scattering, and aerosol absorption enter different terms. For a path of length s , optical depth and direct transmittance have the Beer–Lambert form

$$\tau(s) = \int_0^s \beta_{\text{ext}(l)} dl, \quad T(s) = \exp(-\tau(s)).$$

Here $\beta_{\text{ext}} = \beta_{\text{Rayleigh}}\rho_m + (\beta_{\text{aerosol-scatter}} + \beta_{\text{aerosol-absorb}})\rho_a$, evaluated per RGB channel. Extinction removes light from the original scene color; only the scattering coefficients contribute light into the view. In particular, treating all dust extinction as scattering would make a dense absorbing dust layer too bright. The renderer’s red-dust coefficients and droplet approximation are artistic optical parameters, while the molecular wavelength trend follows the Rayleigh idea.

The pixel combines attenuated scene radiance and light scattered into the view:

$$L_{\text{eye}} = T_{\text{view}}L_{\text{scene}} + L_{\text{in}}.$$

At each view-ray sample, incoming solar radiance is reduced by transmittance along the light path. The source is multiplied by local density and a phase function: Rayleigh scattering uses a symmetric $1 + \cos^2 \theta$ dependence, while aerosols use a forward-peaked Henyey–Greenstein form with fixed asymmetry $g = 0.65$. The source is then attenuated along the remaining view path before reaching the eye and contributes to L_{in} . This is why a long low-Sun path can redden transmitted light while a nearby sky region gains blue molecular scattering. A small indirect fill term approximates light arriving from other directions; it is not a multiple-scattering solve.

The aerosol state responds to configured dust and water. The water partial pressure is compared with a temperature-dependent saturation estimate; excess vapor can become suspended liquid according to a configurable fraction. A simple droplet cross-section gives an extinction estimate from liquid mass and droplet radius. Thus cooling at fixed water input can increase cloud or mist opacity in the model. It does not simulate nucleation, droplet-size distributions, precipitation, or convective transport. A weather model could supply local pressure, temperature, humidity, and liquid water to the optical layer; the current renderer consumes one configured reference state.

Terrain and the planet also control which sample receives direct Sun light. A spherical horizon test blocks the central solar ray below the horizon; a finite solar angular radius softens the twilight transition. Away from that narrow horizon region, the terrain shadow map can block sunlight from a sample near a mountain. The finite-disk transition is an approximation rather than an integration over the Sun’s actual disk, and shadow-map resolution limits the detail of mountain silhouettes. Without the terrain query, shadowed air would continue to glow as if its light path passed through a mountain.

The implementation uses bounded midpoint samples for the view integral. Within a sample it integrates constant source and extinction analytically through $\frac{1 - \exp(-\beta_{\text{ext}} ds)}{\beta_{\text{ext}}}$, avoiding an excessive bright contribution in thick dust. A precomputed multiple-scattering atmosphere, such as the approach in R2, would describe high-order skylight and horizon brightening more faithfully. It would require larger lookup tables and careful parameterization when pressure, temperature, dust, water, or geometry change interactively. The present single-scattering calculation keeps those controls responsive and its cost bounded.

5.3. Why refractive rays bend

The same pressure, temperature, and gas composition produce an approximate surface refractive index. At fixed pressure, cooler air is denser, increasing $n - 1$ in this model. The radial profile decreases with altitude and tapers to $n = 1$ at the shell boundary. Fermat’s principle says a ray follows a path of stationary optical

length; in a smoothly varying index, its direction changes according to the perpendicular component of the index gradient:

$$\frac{d(d)}{ds} = \frac{\nabla n - (d)((d) \cdot \nabla n)}{n}.$$

Only the gradient perpendicular to the ray bends it; a gradient along the ray changes optical travel time but not direction. A midpoint step follows this curvature, and the radial symmetry gives a useful check: the spherical Snell quantity $nr \sin \theta$ should remain approximately constant. CPU tests check this invariant and ray reciprocity; GPU tests compare reprojected sky displacement with the CPU reference and check cold-versus-warm configured scenes. Figure 10 separates ray geometry from scattered radiance.

Atmospheric density, curved sightline and scattering

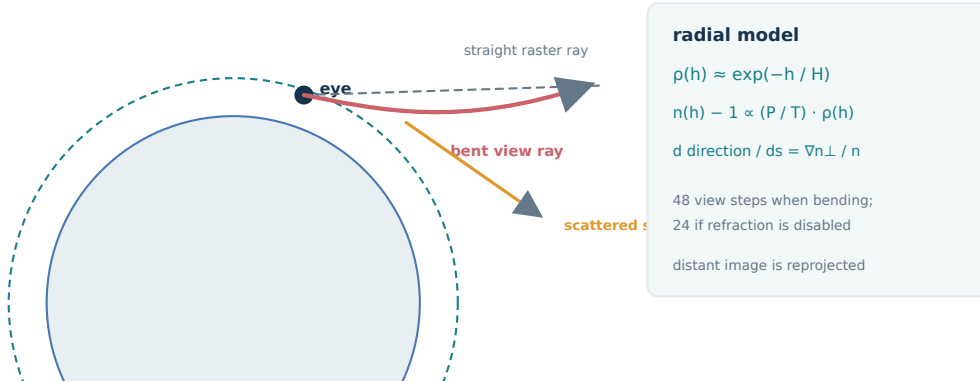


Figure 10: A refractive view ray bends through denser air while solar light contributes scattered radiance. Near terrain keeps its rasterized silhouette; only visible distant imagery is reprojected. Generated schematic.

The renderer bends a view ray and reprojects a visible distant image pixel when that ray escapes the shell. Near terrain retains its rasterized silhouette, and direct-light shadow rays remain straight. An off-screen or hidden source cannot be reconstructed from the existing color buffer; trapped rays do not leak background light. Full curved-ray tracing through scene geometry would represent refracted terrain and solar visibility more consistently, but would require a different visibility structure and substantially more ray queries. The pressure and temperature dependence has a physical basis (R3), while the chosen constants and miniature-world scale remain a rendering model rather than precision terrestrial refractometry.

A deliberately extreme test compresses the shell to 15 m around the 1 km planet, sets pressure to 150,000 Pa and temperature to 180 K, and places the surface camera 2 m above the ground with a 15-degree field of view. The two actual renderer captures below differ only in the JSON `refraction_enabled` switch. Refraction extends the visible solar limb into sky pixels near the horizon, while the already rasterized Sun and terrain keep their silhouettes. A small pointed edge remains where terrain hides the source image; this is a limit of image reprojection rather than a prediction of solar optics. The setting illustrates the rendering model's behavior, not Earth's atmosphere. The replay sidecars in `docs/captures/replay` preserve the exact resolved scenes.

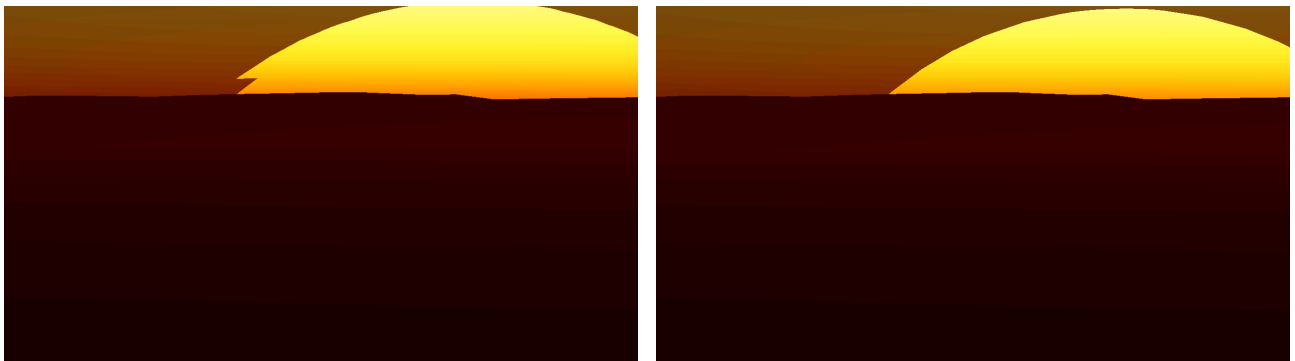


Figure 11: Matched full-resolution renderer captures of extreme refraction enabled (left) and disabled (right). Only the JSON bending switch changes; scattering and exposure remain active in both.

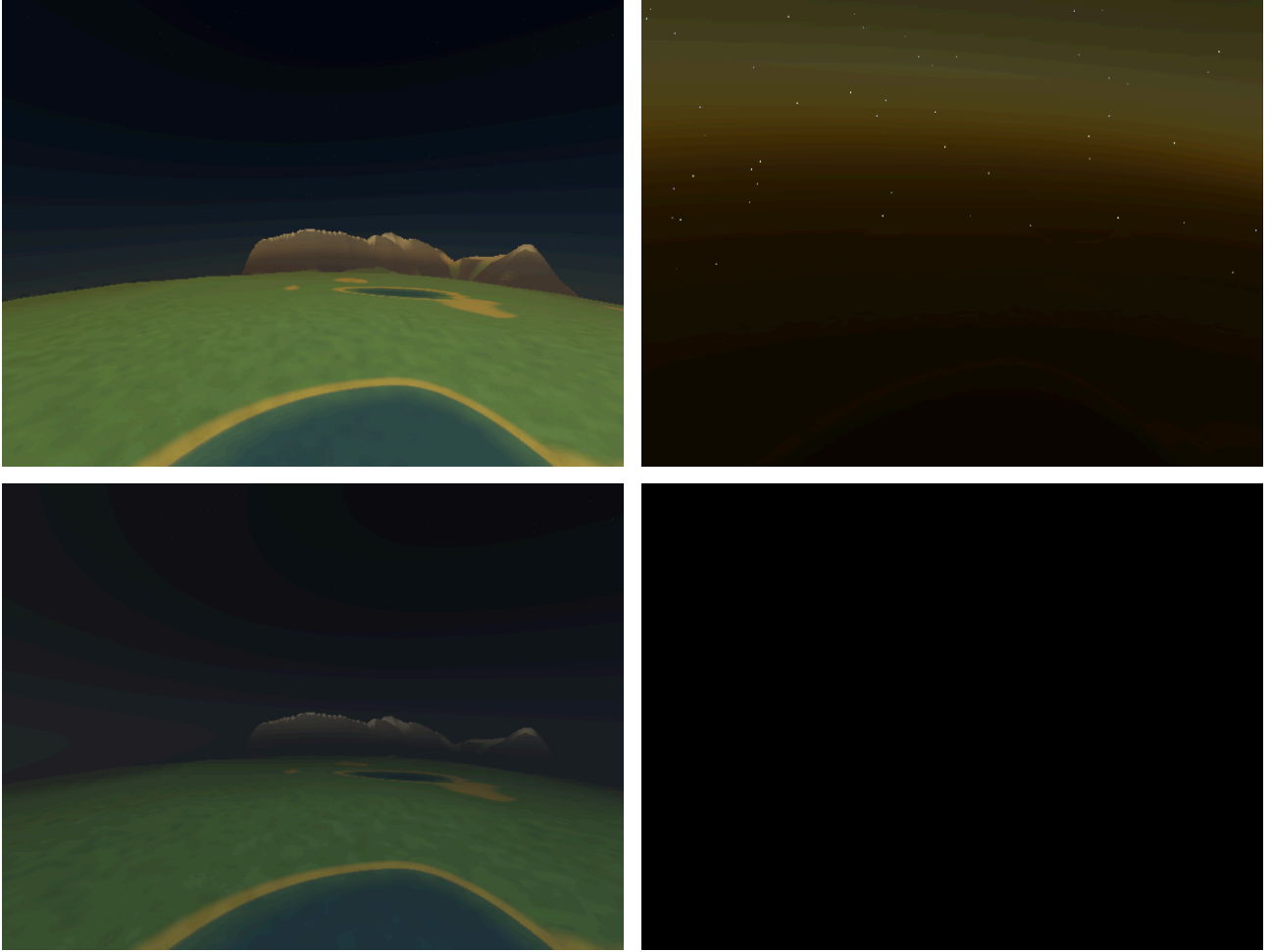


Figure 12: Current controlled atmospheric scenarios, left to right and top to bottom: standard air, sunset, condensed mist, and heavy red dust. All four are actual renderer captures from the regression gallery.

5.4. Where the atmospheric computation is spent

For each visible pixel, the view ray needs repeated density and light-path evaluations. A reusable GPU density-column table stores the geometric integrals of molecular and aerosol density along straight light paths. Since the table stores columns rather than color, changing the gas mixture or dust coefficients can reuse the same geometry and apply new extinction coefficients. Camera motion also leaves the shell geometry unchanged. The main atmosphere field normally uses quarter resolution; a depth-aware resolve reconstructs smooth regions and reintegrates full-resolution pixels near discontinuities. Terrain, stars, and the Sun stay at full resolution. This matches the physical structure of the calculation: smooth optical fields tolerate coarse sampling, while silhouettes and depth edges do not. Figure 13 summarizes the pipeline.

An alternative is full-resolution ray integration for every pixel. It avoids resolve artifacts, but repeats nearly identical work across neighboring sky pixels. A larger multidimensional lookup could reduce per-pixel work further, but would be harder to update for changing aerosol and humidity states. The current split keeps the expensive geometric column reusable while leaving composition as a live control.

Atmospheric composition: reusable columns and depth-aware fields

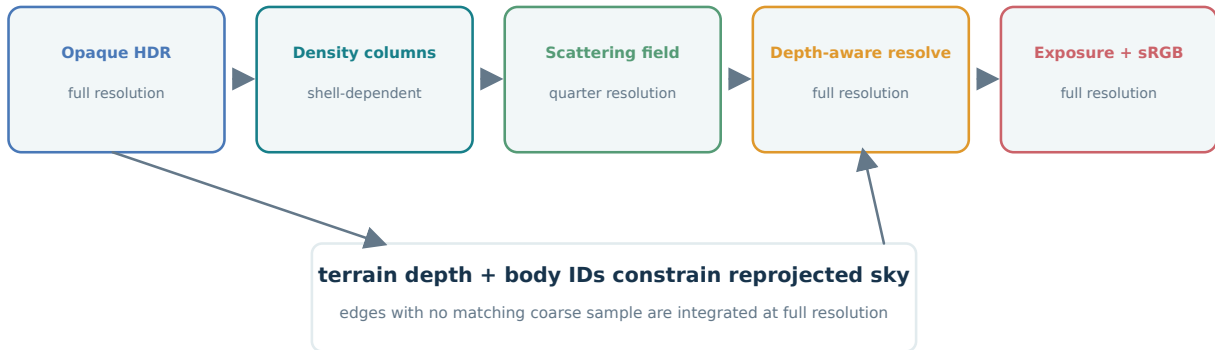


Figure 13: Atmospheric rendering graph. A reusable density-column lookup and a reduced scattering field lower repeated work; full-resolution depth controls edge reconstruction. Generated schematic.

The project’s recorded Quadro M1000M benchmark at 1280×720 reported 208 ms mean frame time for the original full-field method, 138 ms with the column lookup, and 26 ms with reduced fields and shared ray calculations. These were 90-frame measurements of one surface scene with Debug CPU builds, recorded in the README; Figure 14 reproduces them rather than claiming a new experiment. The approximately eightfold change is specific to that workload and GPU. A separate RelWithDebInfo run reported 28.5 ms. A headless software-rendered capture is not comparable to this hardware benchmark.

Quadro M1000M benchmark reported in the project README

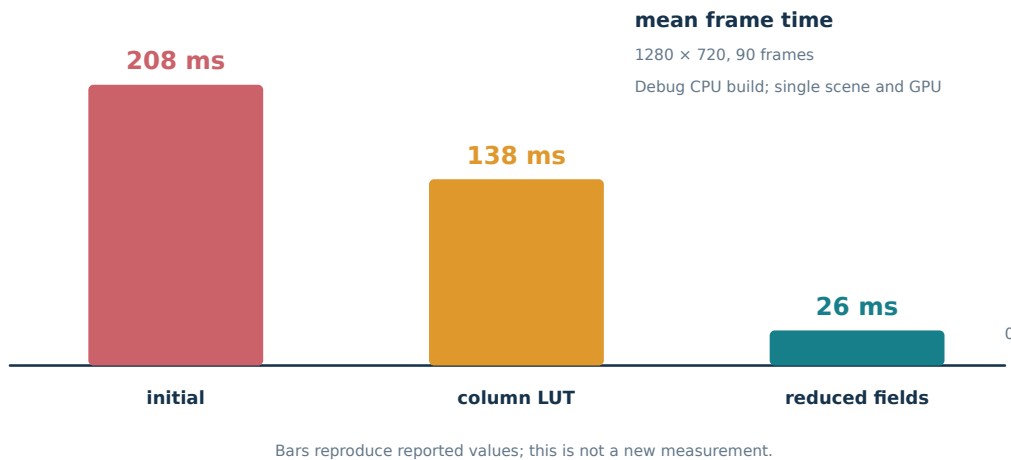


Figure 14: Reported mean frame times from README.md, Quadro M1000M, 1280×720 , 90 frames. These bars visualize previously reported measurements, not a new benchmark.

A profiler now records CPU stage times and asynchronous GPU timestamp results without blocking for each query. CPU frame time, GPU pass time, and swap/presentation delay answer different questions; adding CPU and GPU times would double-count overlap. A completed HDR scene may be reused if camera, time, viewport, geometry, and pending-terrain state match. The HUD remains separately drawn so input can reveal or hide it immediately.

5.5. Perlin wind on a rotating planet

Grass wind now samples three advected 3D Perlin gradient fields, replacing the earlier overlapping sine waves. The field input is each root in body-local metres, so orbital translation and rotation do not change the local deformation. Gust cells span 12.5 m, direction cells 50 m, and flutter cells approximately 1.4 m. The implementation blends gradient dot products at eight cell corners with the quintic fade $6t^5 - 15t^4 + 10t^3$, following the construction in Ken Perlin’s improved-noise reference. Integer hashing selects corner gradients instead of a permutation table. This is a visual wind model, not a fluid simulation.

The lattice repeats every 256 cells. Drift rates are binary fractions of a cell per second with a common 8192-second period, allowing the double simulation clock to wrap before conversion to a float uniform without a discontinuity. Both the main and water-reflection pass use the same field and phase. Wind strength zero leaves the seeded static lean; pause still freezes wind, preserving deterministic captures. The vertex shader adds no textures, triangles or per-instance uploads, but performs more arithmetic than the old sine field. GPU transform-feedback checks exercise lattice and period boundaries, both poles, large wrapped times, fixed blade roots and independence from body transforms; pixel tests retain wind motion, shadowing and reflection clipping checks.

5.6. Grass geometry budgets and eight distance levels

The glvertexid heightmap example reconstructs regular-grid coordinates from vertex IDs and connects strips with degenerate vertices. Its published example contains neither eight LODs nor sinking; those requested techniques are adapted here to the existing instanced grass. Terrain retains its irregular spherical tessellation and shared shoreline edges, which cannot be represented directly by that regular height-only grid.

Blade strips now terminate in one tip, removing the former duplicate tip vertex and degenerate triangle. Eight nominal distance levels use 6, 5, 4, 3, 2, 1, 1 and 1 segments. At the working 60 m draw range, boundaries are 7.5, 9.375, 11.25, 13.125, 15, 30 and 45 m. Geometry straightens between 3.75 and 7.5 m, before any segment reduction can occur; removing collinear intermediate vertices preserves its shape. The CPU subtracts the 9 m movement allowance when choosing detail, keeping enough geometry while the cached patch follows the walker. Each retained blade submits no more vertices than the previous near/far scheme.

Eight deterministic retention groups thin distant candidates further, retiring one eighth on average in each equally spaced interval from 15 to 60 m. Smooth sinking and collapse update in the shader on every frame. CPU rejection uses the same group hash and adds the movement allowance, so a discarded root cannot become visible before the next rebuild. This deliberately trades distant density and shape detail for less work; the near-camera Gaussian candidates and terrain remain unchanged. Both main and reflection passes use the same decisions. One sorted instance buffer replaces two separate buffers. Compatible one-triangle levels merge into a single draw, leaving at most six instanced draws per pass rather than two previously, but only one instance upload per rebuilt patch.

Initial frozen grass view	Previous	Eight LODs
Uploaded blade instances	162,828	146,510
Vertices submitted per pass	1,477,562	1,081,766
Triangles submitted per pass	1,151,906	788,746
Instance bytes per rebuild	6,513,120	5,860,400
Instanced draws per pass	2	6

These counts come from the same generated roots: vertex submissions fall by 26.8%, submitted triangles by 31.5%, and instance payload by 10.0%. Triangle counts include submitted degenerates and fully sunk reserved geometry; bytes exclude driver overhead. CPU tests check guard-band visibility and the vertex bound. GPU transform feedback checks endpoint agreement, sinking continuity and distant shape invariance; a primitive query verifies submitted triangle accounting. Full timings, raw traces, executable and shader-tree hashes, frozen inputs and reproduction commands are in `docs/journal/benchmarks/grass-lod.md` and its data directory.

A paired 640×360 walking capture on Mesa llvmpipe with two worker threads measures 20 frames, excluding three warmup frames and the final readback frame. Mean frame time falls from 1576.97 to 1463.22 ms (7.2%); the median falls from 1387.17 to 1295.95 ms (6.6%). The foliage-disabled control changes from 708.99 to 720.05 ms mean, and grass preparation per rebuild remains similar. This single software-renderer pair measures a quality/performance tradeoff, not a hardware FPS guarantee. Both versions rebuild grass six times and install three terrain meshes in the measured window; bare final captures match exactly.

5.7. Camera movement and grass preparation: measured costs

The following measurements predate the Perlin wind change; their recorded executable hashes and captures identify the earlier sine-wind workload.

On 2026-09-30, a fixed-step benchmark separated the cost of moving the surface camera from orbital motion. Three cases share the same camera and scene: stationary with paused time, walking with grass, and walking without grass. Each capture has 36 frames at 640×360; five warmup frames and the final readback frame are excluded. Walking advances 2 m per frame. The working scene uses a 60 m grass draw distance and a 200,000-

blade budget. The host runs Mesa 22.3.6 llvmpipe with two worker threads under Xvfb; CPU code uses GCC 12.2.0 with `-O2`. These are software-renderer measurements, not a reproduction of the reported 30-to-10 FPS change on a physical GPU.

Case	Frame mean / median (ms)	GPU passes mean (ms)	Grass per rebuild (ms)
Before: stationary	36.64 / 36.59	2.09	0
Before: walking	1089.03 / 872.44	924.25	79.92
Before: walking, bare	620.87 / 478.28	481.32	0
After: stationary	37.26 / 36.96	2.20	0
After: walking	1033.55 / 869.35	878.71	50.79
After: walking, bare	639.14 / 482.17	498.60	0

Both walking runs rebuild grass on ten measured frames and install five terrain meshes. All thirty stationary frames reuse the completed scene. Captures construct terrain synchronously for reproducible geometry; live walking already constructs land and water in the background. The optimized capture’s CPU mesh stage averages 136.34 ms per frame, with a median near zero. This average must not be interpreted as a render-thread stall in the interactive path. CPU and GPU times overlap, and swap time includes software-driver waits.

Two local changes reduce grass preparation without changing its roots or appearance. A conservative triangle bound rejects candidates whose random sample cannot pass the Gaussian density test, before computing their full geometry. Sorting caches each squared distance once, then sorts compact keys and gathers the instance buffers. Mean placement time on rebuilding frames falls from 55.59 to 35.65 ms, sorting from 23.51 to 14.26 ms, while instance submission stays near 0.85 ms. Complete preparation falls from 79.92 to 50.79 ms, about 36%. Three isolated placement runs per version, alternating order and discarding two warmup iterations per run, reduce median process CPU time from 52.505 to 34.6775 ms (34%). All iterations retain exactly 162,828 blades and identical attribute checksums. All three paired final PNGs are byte-identical.

The median walking frame remains nearly unchanged, at 872 versus 869 ms. Rendering grass, reflected geometry and atmosphere dominates this software-renderer workload. Grass is already instanced and distance-cached; it is drawn in both opaque and reflection passes. Partial patch updates could reduce preparation spikes further, but would not eliminate those draw costs. They also require stable terrain tile identities: candidate seeds currently depend on triangle indices, which can change with tessellation. Reusing roots across mesh revisions without resampling could leave grass above or below the new ground. A future tiled cache or background placement job should retain overlap, rebuild changed tiles and discard obsolete jobs. The later culling and LOD work should measure each render pass on the target GPU before choosing a broader redesign.

Reproduction commands, complete per-pass tables, frozen inputs, raw CSVs, executable hashes and caveats are retained in `docs/journal/benchmarks/camera-movement.md` and its `movement/` data directory. Live traces now expose foliage placement, sorting, upload and rebuild counts separately. No end-to-end hardware FPS improvement is claimed from this single paired software-rendering run.

5.8. Nested CPU tracing and function-level profiling

The optional `--cpu-trace` records nested scopes with elapsed wall time and, where available, calling-thread CPU time. Render and terrain-worker lanes are separate. The standalone HTML report provides expandable caller trees, a chronological SVG timeline, self-time rankings, and reversed callee-to-caller paths. Self time subtracts immediate children; it still includes uninstrumented calls, driver work, waits, and tracing overhead. Inclusive times must not be added across ancestors and descendants. Recording begins in `Renderer::run`, after constructor startup.

A frozen 60 m grass scene was measured on 2026-09-30 with GCC 12.2, Xvfb and two llvmpipe workers, at 640 × 360 pixels. Fourteen frames advance the walking camera by 2 m per frame with simulation time paused; the first three and final frame are excluded. The ten retained walking frames average 4216.18 ms, versus 4179.30 ms with CPU tracing disabled; their final PNGs are byte-identical. This single pair includes host variation and does not establish an overhead bound. Stationary paused-image reuse averages 131.36 ms; walking without grass averages 2021.34 ms. These are software-renderer diagnostics, not hardware FPS measurements.

Scope over ten walking frames	Wall (ms)	Thread CPU (ms)
Grass draws, main + reflection	16059.27	16050.88
Atmosphere, main + reflection	18452.14	36.81
Terrain construction	4313.86	4312.70
Shoreline refinement (inside terrain)	3683.14	3682.22
Grass placement	421.21	421.17
Grass LOD batching	150.42	150.37
Grass upload	7.99	7.99

Grass draws consume about 38.1% of scoped frame wall time and 72.1% of the render thread’s CPU time, including software OpenGL work. Atmosphere takes 43.8% of frame wall time but little calling-thread CPU time, indicating blocking or scheduling while other driver workers may execute. Terrain construction contributes 10.2%; shoreline refinement is 85.4% of terrain time. Placement, batching and upload together contribute 1.4%, about 145 ms per rebuild. Capture terrain construction is synchronous; the interactive path uses a worker, so capture costs are not interactive render-thread stalls.

The independent Callgrind run records 7,473,993,658 instructions for one terrain build followed by twelve placement/batching iterations, including benchmark checksums. Its exported self and caller/callee reports identify placement, terrain noise, shoreline ordered-map comparisons, rounding and sorting as construction costs. Instruction shares are not time shares; optimized source attribution can split one function across several file entries. This focused CPU workload excludes rendering. Kernel sampling was restricted, and GNU gprofng warned that its interval timer changed; its inconsistent sampled times were rejected. A suitable target-GPU host is still needed for reliable function-level sampled CPU times and hardware optimization conclusions.

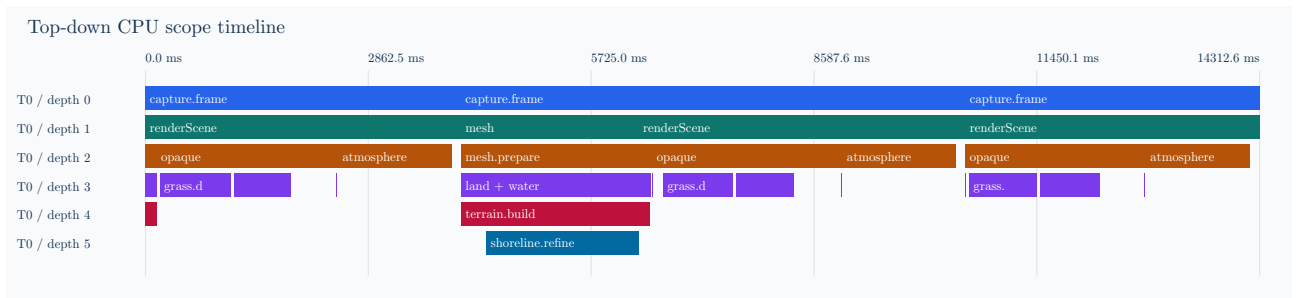


Figure 15: Top-down nested CPU scope timeline for three frozen walking frames on llvmpipe. Width represents wall time, including waits.

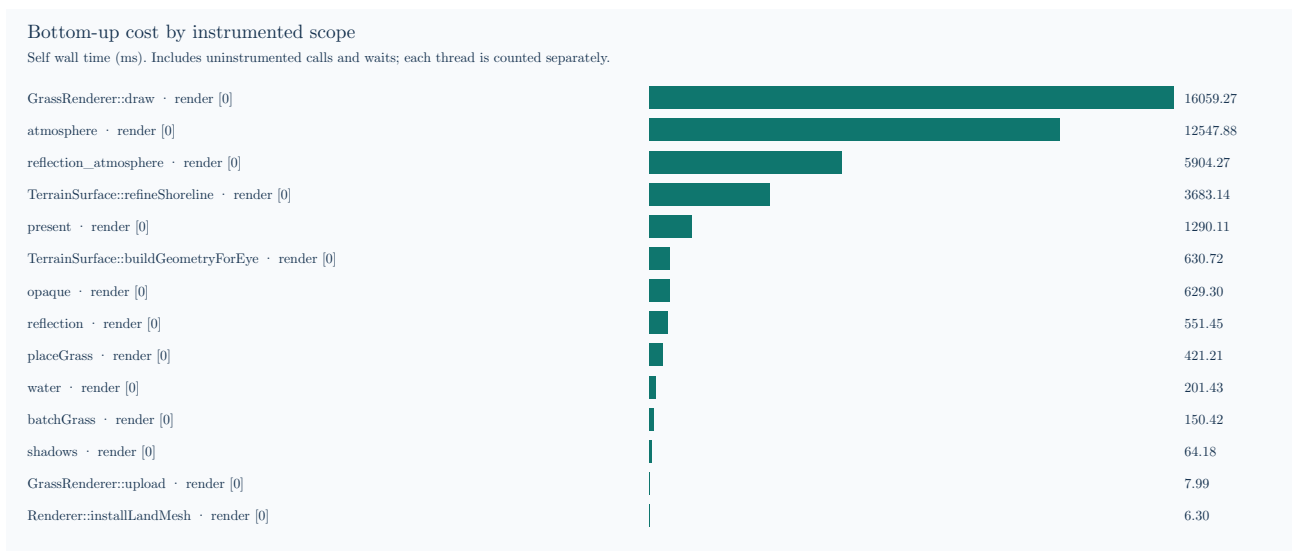


Figure 16: Bottom-up self wall time over ten walking frames. Immediate child scopes are subtracted; uninstrumented work and waits remain.

The retained study is described in `docs/journal/benchmarks/cpu-profiling.md`. It motivates investigating rendered grass/reflection work, atmosphere execution and shoreline construction before assuming that partial placement updates solve the frame-time problem.

6. Visibility, exposure, and interactive constraints

6.1. Diagnostics, dependency boundaries and adaptive quality

The I/Tab panel reports timings and optional Linux NVIDIA NVML utilization. Utilization is sampled off the render thread and is device-wide, while GPU pass milliseconds belong to this application’s measured passes. Software rendering and ambiguous GPU matching show N/A; a fake driver library tests the ABI path without claiming validation on NVIDIA hardware. Figure 17 is a real captured panel.

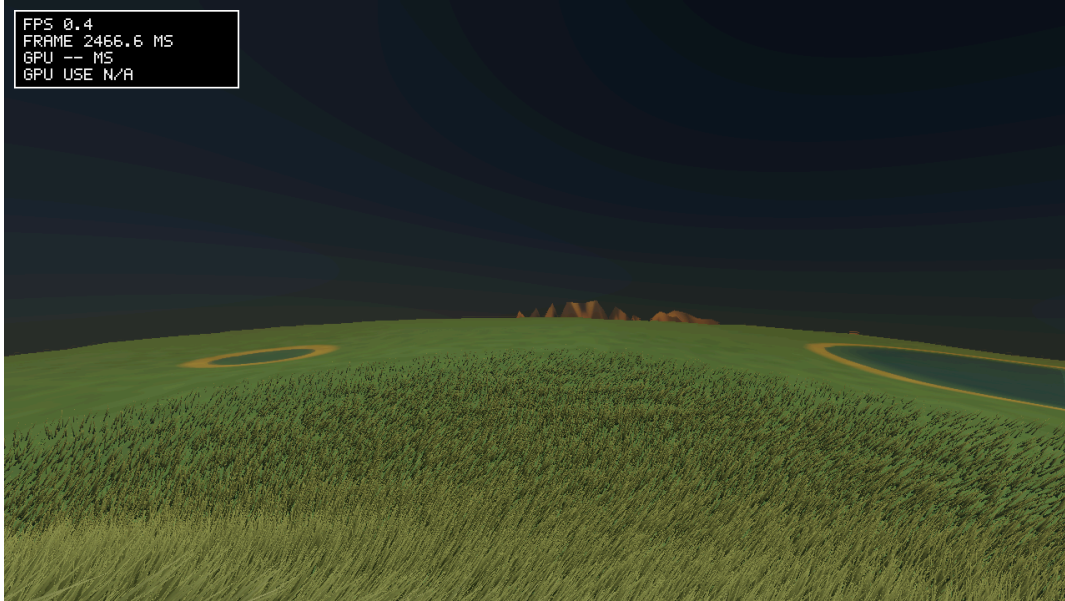


Figure 17: Actual current performance overlay under Xvfb/Mesa llvmpipe. The white-bordered black panel reports FPS, frame time, GPU pass time, and utilization availability; its numbers are capture-environment observations.

A best-effort 20 FPS quality target uses a smoothed frame estimate $m_t = 0.85m_{t-1} + 0.15f_t$ to select one of six scene-resolution scales, from 100% down to 25%. After a 1.5-second cooldown, sustained times above 55 ms lower resolution; times below 32 ms permit recovery. Only moving interactive scenes feed the controller. A graphics-memory estimate can cap the initial scale, reserving most reported memory for other uses; fixed-resolution captures remain deterministic. The HUD stays at native window resolution. Figure 18 shows the control law, not a measured response trace. This is a rendering control, not a change to the atmosphere’s physical parameters. Its lower resolution may still lose thin clouds or distant silhouettes; a strict 20 FPS guarantee is impossible once the minimum scale is reached or scene cost is unrelated to pixel count.

Best-effort quality control: timing feedback and memory cap

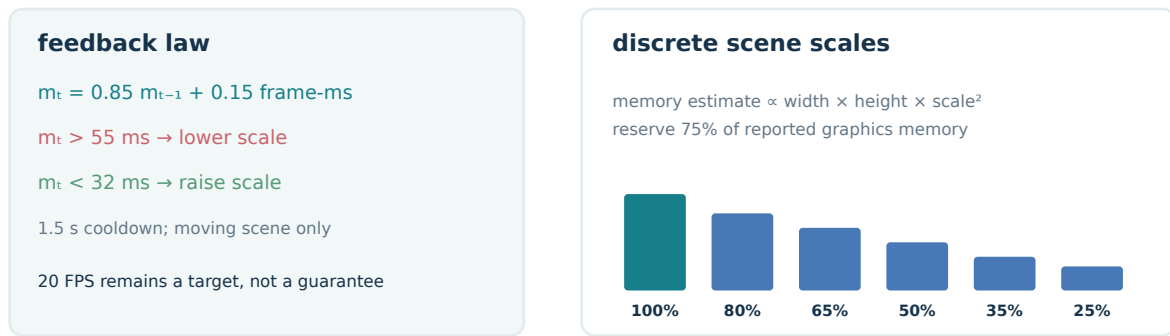


Figure 18: Actual controller thresholds and discrete scene scales; bar heights are relative scale, not measured frame rates. Generated schematic.

6.2. Visibility must follow the light path

The same visibility principle applies to air and water. A mountain blocks direct sunlight from nearby air, so the direct in-scattering term uses the planet's terrain depth map. The finite solar disk's twilight region remains, as does the approximate indirect fill. For water, a valid bright sample in the reflection image does not prove that a water point is lit: that point may lie on the night side or behind terrain. The direct reflection term therefore requires positive local Sun incidence and terrain visibility. Indirect water radiance remains. Figure 19 illustrates the distinction. The gallery's terrain-shadow and twilight cases are controlled visual regressions.

Why a geometric reflection must be gated on the night side

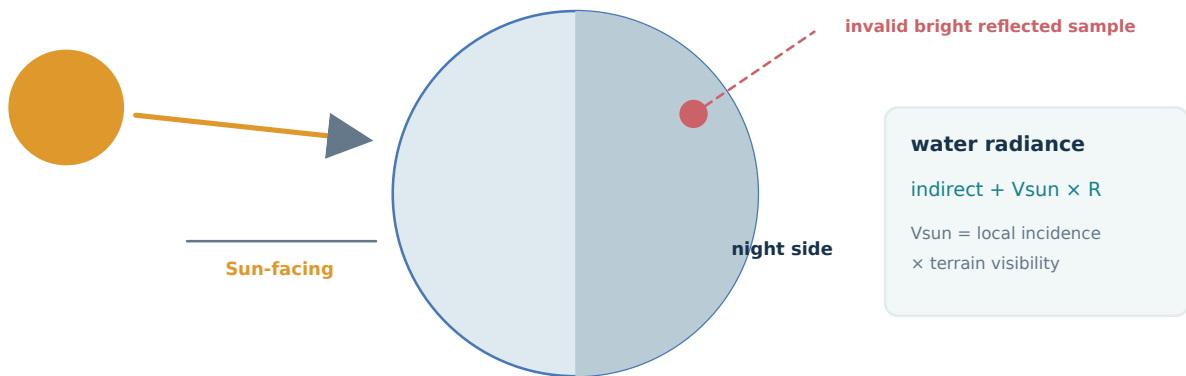


Figure 19: A reflected bright sample can exist in image space even when the water point lies on the night side. Local incidence and shadow-map visibility gate the direct reflection term. Generated schematic.

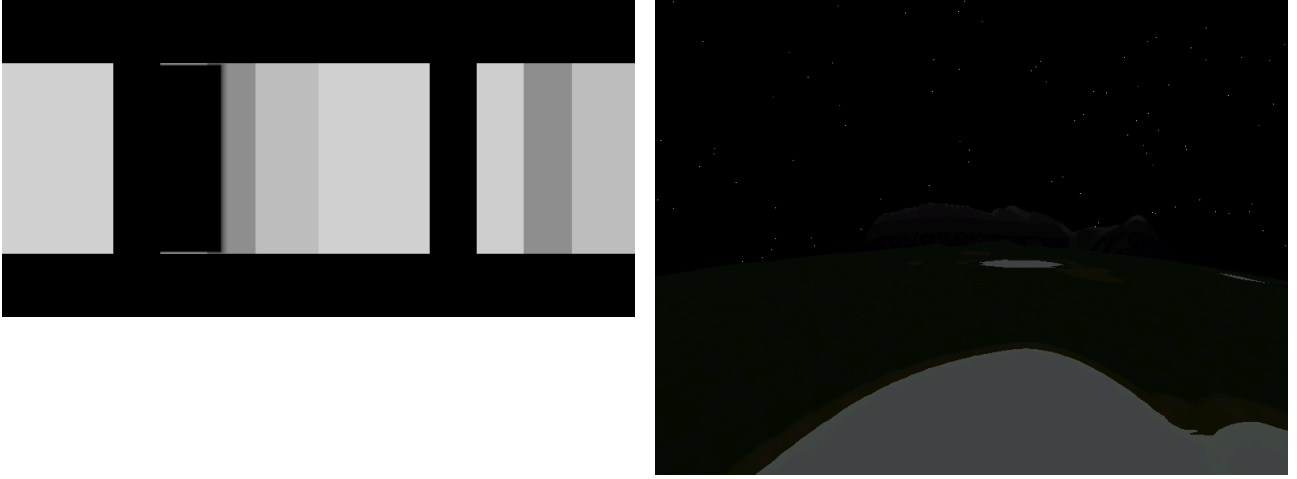


Figure 20: Actual current regression captures for terrain shadow visibility (left) and near-horizon twilight (right). Their fixtures and generation commands are recorded with the gallery.

6.3. Interpreting motion from the camera

Ten-revolution predictive trails and body cards help distinguish local orbits from inertial paths. The path color averages terrain vertex RGB factors multiplied by the body's configured base tint; labels report name, parent, axes, eccentricity, period, radius, and mass. Sampling the entire orbital system, rather than each child's local ellipse alone, is necessary for moving-parent trails. The overlay is toggled by 0 and hidden in planet and surface cameras; real X11 input tests verify both behaviors.

Orbit-to-surface entry is a one-second wall-clock transition. The starting pose is the last actually rendered view, avoiding a first-frame jump after the key event. An eased parameter $q(u) = 3u^2 - 2u^3$ drives spherical interpolation of the planet-relative radial direction, logarithmic interpolation of radius, quaternion interpolation of orientation, and field-of-view interpolation. Intermediate positions are clamped above sampled terrain and water. The target is resampled while the planet moves, so the transition ends at the current surface pose. This is a camera path, not a physically accelerated descent; a dynamics-based flight model would require forces, velocities, and collision response. Figure 21 shows the path and ease curve.

One wall-clock second from orbital pose to surface pose

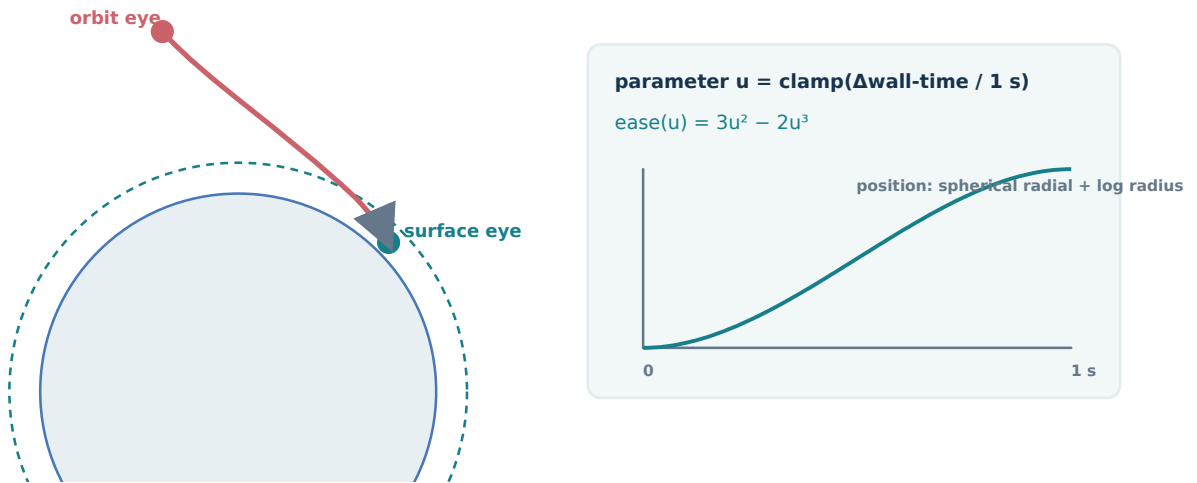


Figure 21: Orbit-to-surface descent. The curve shows the implemented smoothstep parameter; the spatial path is schematic because the actual start, target, and terrain vary by scene.

7. Evaluation and model boundaries

7.1. What the evidence establishes

The repository contains 43 CTest entries covering configuration and orbital invariants; terrain and camera behavior; lighting, shadows, atmosphere and refraction on OpenGL; renderer captures; adaptive quality; and real GLFW input under Xvfb. The 17 README images are checked against their SHA-256 manifest. These

checks establish specific numerical and rendered behaviors on the tested software renderer. They do not establish physical fidelity, portability to every GPU driver, or a guaranteed 20 FPS for arbitrary scenes.

The model boundaries are deliberate. Orbits are prescribed two-body ellipses with hierarchical recoil. Terrain is a deterministic synthetic height field with bounded tessellation. Atmospheric scale heights are chosen relative to shell size; the model does not solve hydrostatic temperature profiles, cloud dynamics, weather, or multiple scattering. Refraction bends view rays and reprojects available distant imagery but keeps near opaque geometry and direct shadow rays straight. Atmospheric temperature is read from JSON rather than evolved from sunlight and orbit. These limits define what a visual comparison can support.

7.2. Rebuilding the paper and the evidence

From the repository root, regenerate the deterministic vector figures and compile the Typst source:

```
python3 docs/journal/make_figures.py
typst compile --root . docs/journal/paper.typ docs/journal/paper.pdf
```

The 12 SVGs require only Python’s standard library. The paper references the versioned PNGs already stored in `docs/screenshots/`. To rerun the application tests in a headless OpenGL environment, build the CMake project with testing enabled and execute `LIBGL_ALWAYS_SOFTWARE=1 xvfb-run -a ctest --test-dir build --output-on-failure`. Exact image-generation commands and checksums are in `docs/captures/generation.json`.

8. Conclusion

The renderer’s most useful physical separation is between the state of a world and the approximation used to draw it. A fixed terrain height field can be sampled at changing mesh densities; prescribed orbital positions can be evaluated at any time; and atmospheric extinction, in-scattering, and refractive curvature can be varied from a configured air state. The atmosphere is strongest as an interactive optical explanation of why density, path length, particle type, and visibility change a sky. Its limits are equally specific: shell-relative scale heights, RGB coefficients, single scattering, fixed reference weather, and image-based refraction prevent a predictive climate or precision optical interpretation.

9. References and implementation map

- R1. NASA, *Basics of Space Flight, Chapter 3: Gravity and Mechanics*, official orbit primer. Used for the classical two-body context; the implementation details above come from `src/simulation/OrbitalSystem.h`.
- R2. E. Bruneton and F. Neyret, “Precomputed Atmospheric Scattering,” *Computer Graphics Forum* 27(4), 2008, doi:10.1111/j.1467-8659.2008.01245.x. The project uses a smaller runtime single-scattering approximation and a limited reusable lookup.
- R3. NIST Engineering Metrology Toolbox, Index of Refraction of Air documentation. Background for pressure, temperature, humidity and refractive index; this project does not claim NIST-grade refractometry.
- R4. IAU 2015 resolutions, official PDF. The rendering code uses the Resolution B2 bolometric zero-point convention for luminosity.
- R5. *PlanetSimulation* repository: implementation in `src/`, shaders in `shaders/`, checks in `tests/`, and captured images with reproducibility metadata in `docs/captures/generation.json`.
- R6. NASA Glenn Research Center, Ideal Gases under Constant Volume, Constant Pressure, Constant Temperature, and Adiabatic Conditions. The hydrostatic, isothermal scale-height derivation is the physical motivation for the exponential profile, not the formula used to set the renderer’s shell-relative scale heights.

9.1. Claim-to-artifact map

Claim	Primary implementation	Validation
Stable terrain under LOD	<code>src/rendering/geometry/Terrain.h</code>	<code>TerrainTests</code> , surface capture
Barycentric orbit hierarchy	<code>src/simulation/OrbitalSystem.h</code>	<code>OrbitalSystemTests</code>
Shadowed atmosphere	<code>shaders/atmosphere/atmosphere.frag</code>	<code>TerrainShadowRenderIntegration</code>
Temperature-controlled bend	<code>src/simulation/Atmosphere.h</code>	<code>AtmosphereTests</code> , GPU render test

Adaptive scene scale	src/rendering/ diagnostics/AdaptiveQuality.h	AdaptiveQualityTests, live render
One-second descent	src/rendering/camera/CameraTransition.h	CameraTransitionTests, X11 input